VITALO OPERATIONS

ЭТАП 5 — UI/UX И ЭКРАНЫ

Версия 1.0 · 06 октября 2026

Статус: проект интерфейса Operations поверх существующего VITALO MVP

> Главная задача: добавить новые операционные функции без ощущения, что пользователь получил другую систему. Существующий визуальный язык VITALO, структура frontend и брендовый цвет сохраняются; новые экраны должны быть продолжением текущего Admin.

1. Что мы строим на UI/UX-этапе

Этап 5 переводит предыдущие документы из «что должно происходить» в «что пользователь видит и нажимает». Это последний большой слой перед сборкой технического задания для Claude Code.

Не строим: новый Admin с нуля, отдельный складской frontend, мобильное приложение Warehouse или тяжёлый WMS.

Строим: расширение существующего Admin + простые task-oriented экраны для Warehouse + новые разделы Operations.

2. UX-принципы

- Существующий VITALO frontend остаётся визуальным ориентиром.
- Брендовый цвет #16A39A используется как accent, а не как заливка всех элементов.
- Один экран — одна основная задача.
- Основная кнопка всегда очевидна.
- Складской пользователь не видит ненужную финансовую информацию.
- Минимум полей, максимум автозаполнения.
- Сканирование предпочтительно; ручной ввод всегда доступен как fallback.
- Каждое критическое действие подтверждается.
- Ошибки объясняются простым языком.
- Повторное действие не должно создавать дубликат.
- Не использовать сложные модальные окна там, где достаточно простой страницы/карточки.
- Desktop Admin остаётся основным рабочим интерфейсом; Telegram используется прежде всего для уведомлений.

3. Информационная архитектура Admin

```
 Dashboard · ⚡ Задачи ·  Заказы ·  Продажи ·  Склад ·  Закупки ·  Товары ·  Аналитика ·  Расходы ·  Маркетинг · ⚙ Настройки
```

Новые разделы должны использовать существующие sidebar, header, typography, buttons, tables, cards и notification patterns настолько, насколько это возможно. Не создавать второй дизайн-системы внутри VITALO.

4. Dashboard

Цель: быстро понять состояние бизнеса.

Верх: период + основные показатели.

```
Заказы · Продажи · Выручка · Прибыль · Средний чек
```

Середина: склад и операционные проблемы.

```
Свободно · В доставке · Ожидает приёмки · Expiry < 30 дней · MISSING · Возвраты
```

Низ: задачи и предупреждения.

```
⚠ 1 Physical ID не найден · ⚠ 2 возврата ожидают приёмки · ⚠ 3 товара требуют синхронизации цены
```

```
Dashboard не должен превращаться в огромный BI-экран. Приоритет — текущая ситуация и исключения.
```

5. Задачи — главный Warehouse UX

Для Warehouse это самая важная страница.

Структура:

```
Сегодня → Новые → В работе → Завершённые
```

Карточка:

```
Тип задачи · Канал · Заказ/Purchase · Товар · Количество · следующий шаг · основная кнопка
```

Пример:

```
Собрать заказ #62713926400 Yandex Коллаген Marine 1 шт. Physical ID: V-000184 [ СКАНИРОВАТЬ ]
```

Warehouse не должен открывать пять страниц для выполнения одной операции. Task ведёт его по цепочке.

6. Экран сканирования

Минимальный интерфейс:

```
Ожидается: V-000184 [ СКАНИРОВАТЬ ] или [ ВВЕСТИ ID ВРУЧНУЮ ]
```

Успех:

```
✓ V-000184 подтверждён [ ПРОДОЛЖИТЬ ]
```

Ошибка:

```
 Неверная банка Ожидается V-000184 Получено V-000191 [ ПОВТОРИТЬ ]
```

7. Приёмка Ozon — UI

Шаг 1 — выбор аккаунта:

Ozon Account 1 Ozon Account 2 Ozon Account 3

Шаг 2 — ожидаемые позиции:

Marine Collagen · SKU MRKT-E0UBQPAB · ожидается 5 [ПРИНЯТЬ]

Шаг 3 — фактическое количество:

Ожидалось: 5 Получено: [4] [ПРОДОЛЖИТЬ]

Шаг 4 — срок годности:

Expiry: [11.2028] Количество единиц: 4 [СОЗДАТЬ PHYSICAL ID]

Шаг 5 — результат:

Создано 4 единицы: V-000181 · V-000182 · V-000183 · V-000184 [НАПЕЧАТАТЬ ЭТИКЕТКИ]

Шаг 6:

✓ Этикетки напечатаны ✓ 4 единицы переведены в In Stock [ЗАВЕРШИТЬ ПРИЁМКУ]

При расхождении система показывает его явно, но не заставляет Warehouse разбираться в закупочной экономике.

8. Purchase / закупки — Admin UX

Здесь уже допустима таблица и финансовые поля.

Фильтры: источник · Ozon Account · поставщик · дата · статус · discrepancy

Карточка Purchase:

- Источник и реквизиты.
- Дата.
- Строки SKU / Qty / Original Unit Price.
- Final Actual Total.
- Allocated Actual Cost.
- Expected / Received / Cancelled.
- Расхождения.
- Связанные Physical Units.

9. Orders — UI

Единый список с переключателем:

```
Все | Yandex | VITALO
Фильтры:
Статус · Дата · SKU · Канал
```

Карточка заказа должна показывать только информацию, относящуюся к его текущей операции.

Yandex:

```
Заказ → товар → Physical ID → сборка → доставка → клиент получил / возврат
```

VITALO:

```
Заказ → оплата → Physical ID → сборка → доставка → доставлен / возврат
```

```
Не показывать в одной карточке весь внутренний backend state. Сложность остаётся на уровне системы.
```

10. Sales — UI

Sales — аналитически-фактический список, а не операционный task screen.

Таблица:

```
Дата | Канал | Заказ | SKU | Physical ID | Цена | Себестоимость | Комиссия | Логистика | Реклама | Прибыль
```

Клик по продаже открывает detail с зафиксированными историческими значениями. Изменение текущей цены Product не должно менять прошлую Sale.

11. Physical Unit — detail

В карточке:

- Physical ID.
- Code 128 / печать.
- Product/SKU.
- Purchase.
- Actual Cost.
- Expiry.
- Current Status.
- Current Order, если есть.
- История движения.
- Связанные события.

Главная кнопка зависит от состояния: например «Принять возврат», «Списать», «Исправить», «Напечатать этикетку».

12. Returns — UI

Возвраты не смешиваем с обычным списком продаж.

Разделы:
Ожидаются → Прибыли → Приняты → Списаны

Карточка возврата:

Заказ · Канал · Physical ID · причина · дата · результат
Для Warehouse:
[СКАНИРОВАТЬ ЗАКАЗ] → найден Physical ID → [ПРИНЯТЬ НА СКЛАД]
Если товар непригоден:
[СПИСАТЬ] → второе подтверждение → Write-off

13. Write-off — UI

Warehouse:

Выбрать/сканировать Physical ID → увидеть товар → [СПИСАТЬ] → подтверждение

Admin/Manager:

Видит причину, историю и финансовое влияние.

Причина не должна превращаться в длинную форму. Для MVP достаточно короткого controlled choice + необязательного комментария.

14. Products — UI

Карточка товара остаётся близкой к существующей карточке Product в Admin.

Внутри логически группируем:
Основное → Локализация → Состав → Фото → Коммерция → Yandex параметры → Остаток

Цены:

Yandex: 180 000 VITALO: 220 000 [СОХРАНИТЬ]
Перед сохранением изменения цены:
☐ Цена уже изменена в Yandex ☐ Цена уже изменена в VITALO

Это сохраняет уже согласованный процесс синхронизации цен.

15. Analytics — UI

Аналитика не должна копировать таблицу Sales. Она должна давать вывод.

Первый экран:

Период · Канал · Категория · SKU
KPI:
Выручка · Продажи · Себестоимость · Прибыль · Маржа · Средний чек
Ниже:
Топ SKU по прибыли Товары с низкой маржой Каналы Остатки Сроки годности Замороженные деньги

Вторая стадия может добавить price-impact preview: новая цена → новая маржа → новая прибыль.

16. Expenses — UI

Admin-only.

Добавить расход:
Дата/период · Категория · Канал · Сумма · Комментарий · [СОХРАНИТЬ]

Для рекламы допускается расход за период. Отдельно хранится от комиссии/логистики Yandex.

17. Settings — UI

Минимальная структура:
Сотрудники · Роли · Категории · Бренды · Производители · Страны · Формы · Единицы · Расходы · Печать · Telegram · Action Log

Не перегружать Settings параметрами, которые пользователь никогда не меняет.

18. Desktop / responsive стратегия

- Admin оптимизируется прежде всего под desktop/laptop, где выполняется основная работа.
- Task screens должны оставаться usable на планшете/небольшом экране.
- Таблицы на узких экранах не должны ломать весь интерфейс; допускается локальный горизонтальный scroll.
- Критические кнопки должны быть крупными и однозначными.
- Не использовать hover как единственный способ обнаружить действие.

19. Состояния интерфейса

Каждый операционный экран должен иметь четыре базовых состояния:

```
Loading → Ready → Success → Error
```

```
Для списков дополнительно:
```

```
Empty → Data → Filtered Empty → Error
```

Пример:

«Нет задач на сегодня» — это нормальное состояние, а не ошибка.

20. Подтверждения

Не каждое нажатие требует modal confirmation. Подтверждение нужно там, где действие необратимо или влияет на деньги/остатки.

Без дополнительного подтверждения:

- Открыть карточку.
- Открыть задачу.
- Перейти к следующему шагу.
- Запустить сканирование.

С подтверждением:

- Write-off.
- Correction, меняющая склад.
- Принятие расхождения.
- Подтверждение критического возврата.
- Изменение чувствительной настройки.

21. Главное UX-правило для семьи

```
Если жена или родители должны спросить: «А что мне сейчас нажать?» — экран спроектирован неправильно.
```

Правильный экран показывает:

```
1) Что нужно сделать. 2) С каким товаром/заказом. 3) Какой Physical ID ожидается. 4) Одну главную кнопку. 5) Что произошло после нажатия.
```

22. Связь UI с backend

Каждая кнопка должна соответствовать конкретной бизнес-операции backend, а не просто менять состояние frontend.

Например:

```
[ ПРИНЯТЬ НА СКЛАД ] → backend validates Return → validates PhysicalUnit → creates StockMovement → updates status → writes ActionLog → completes Task → sends notification if required.
```

Это принципиально важно для Claude Code: UI не должен становиться самостоятельным источником бизнес-логики.

23. Что мы фиксируем после Этапа 5

- Существующий VITALO visual language сохраняется.
- Operations добавляется в существующий Admin.
- Warehouse получает простой task-oriented UX.
- Операционные экраны используют последовательные действия и подтверждения.
- Финансовые данные скрыты от Warehouse.
- Telegram — уведомление, Admin — рабочая среда.
- Один источник истины для каждой сущности.

- UI вызывает backend operations; бизнес-правила не дублируются в frontend.

24. Следующий этап

После Этапа 5 архитектура уже описана на пяти уровнях: что строим → где находится → как работает → какие данные хранит → как выглядит.

Следующий шаг — Этап 6: Master Technical Specification для Claude Code. Перед его финальной сборкой будет проведён короткий Q&A; только по тем решениям, которые ещё реально влияют на реализацию. После этого документы можно объединить в единое ТЗ для Claude.

Этап 5 зафиксирован.